GroveNet: Hybrid LiDAR Perception for Crop-Row Navigation in Agricultural Robots

Nurtay Albanbay [1,†], Madina Alimova [2,3,*], Yerik Nugman [4,5,†], Azamat Mustafa [4,5,*], Mukhagali Sagyntay [4,5,*], Rustem Kaiyrov [6], Tansaule Serikov [7], Yerlan Ayapov[1] and Yerlan Koptleuov[4]

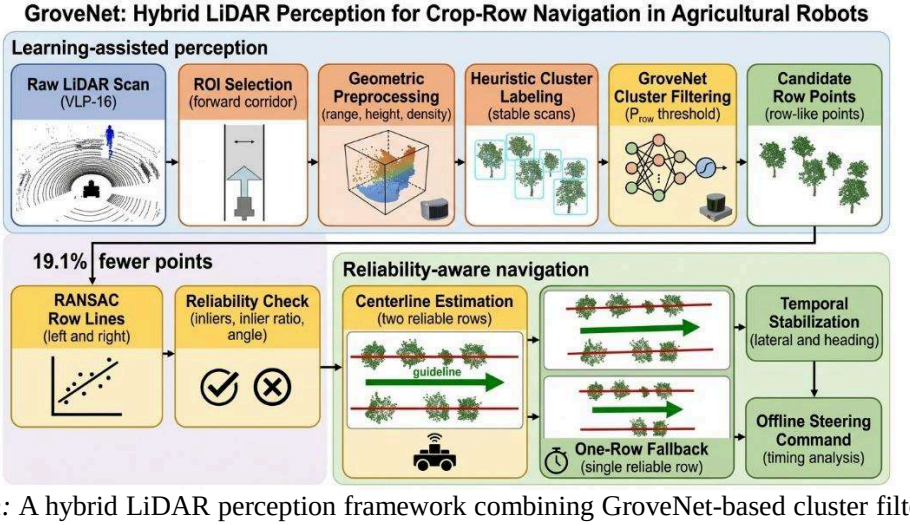
[1] *Cybersecurity and Artificial Intelligence Research Laboratory, Institute of Automation and Information Technologies, Satbayev University, Satbayev 22, Almaty 050013, Kazakhstan*
[2] *Institute of Telecommunications and Automation, Department of Telecommunication Engineering, Almaty University of Power Engineering and Telecommunications, Baitursynuly Street 126/1, Almaty 050013, Kazakhstan*
[3] *Higher School of Telecommunications, Turan University, Satbayev 16a, Almaty 050013, Kazakhstan;*
[4] *Department of Mechanical Engineering, Satbayev University, , Satbayev 22, Almaty 050013, Kazakhstan*
[5] *RnD Center LLP, 050013 Almaty, Kazakhstan*
[6] *Laboratory of Applied Mechanics and Robotics, Buketov Karaganda National Research University, 100001 Karaganda, Kazakhstan;*
[7] *Department of Electronics and Telecommunication, Saken Seifullin Kazakh Agrotechnical Research University, 62 Zhenis Str., Astana 010011, Kazakhstan*
[†] *These authors contributed to this work equally*
* *Correspondence: m.alimova@aues.kz (M. Alimova.), a.mustafa@satbayev.university (A. Mustafa.), mukhagali.sagyntay@gmail.com (M. Sagyntay)*

Abstract

Reliable crop-row navigation depends on consistent row-geometry estimation, which can be hindered by sparse and noisy LiDAR observations. This study presents a hybrid LiDAR pipeline that combines cluster filtering using the lightweight GroveNet model with RANSAC row-line fitting and geometric reliability assessment. Using LiDAR data from the CitrusFarm Dataset, GroveNet achieved a test-set F1-score of 0.9768. Learned filter-ing reduced the mean candidate point count by approximately 19.1% and increased the number of accepted row lines by approximately 25.9% under the same reliability criteria. The proportion of straight-driving scans with at least one accepted row line increased from 70.01% for the geometric baseline to 86.09% for the full pipeline. Validation against a separately constructed reference based on georeferenced canopy geometry and RTK-derived robot positions yielded a mean absolute lateral error equivalent to ap-proximately 2.3% of the inter-row spacing. The mean processing time was 99.93 ms on an NVIDIA Jetson AGX Orin, below the scan interval of the Velodyne VLP-16 LiDAR oper-ating at 10 Hz. These findings demonstrate the value of combining learned filtering with geometric estimation to improve the availability of accepted row geometry while main-taining computational feasibility for onboard agricultural navigation.

Keywords: LiDAR-based navigation; Crop-row detection; Autonomous agricultural robots; Learn-ing-assisted filtering, GroveNet.

Table of Contents



Innovative Description: A hybrid LiDAR perception framework combining GroveNet-based cluster filtering with reliability-aware geometric row estimation for agricultural robot navigation is reported.

1. Introduction

Agricultural robotics has become an important focus in precision agriculture be-cause modern farms require higher productivity, reduced labor dependence, and more selective use of resources. Mobile robots are increasingly used for crop monitoring, spraying, harvesting, fruit mapping, and targeted weed control, and they must operate repeatedly in structured yet variable field environments.[1-6] For these operations, nav-igation between crop rows is not only a mobility challenge; it is a prerequisite for stable sensing, safe motion, and accurate execution of field operations.

Autonomous crop-row navigation has been investigated using a range of sensing and control approaches, including vision and depth sensing.[7] Vision-based methods detect crop rows and estimate guidance lines for row following.[8-10] These visual estimates can support motion control through fuzzy logic or image-based visual servoing.[11-13] Hybrid navigation frameworks integrate GNSS, compass information, and machine vision to combine within-row guidance with field-level routing.[14] These stud-ies show that agricultural navigation is strongly linked to row-detection quality, sensor reliability, and the control system's ability to translate perceived row structure into stable steering commands.

LiDAR is relevant to crop-row navigation because it provides direct range meas-urements of surrounding geometry and is less sensitive to illumination changes than passive vision sensors.[15] Previous studies have extracted crop-row lines from 2D Li-DAR scans and used the resulting geometric models to guide agricultural robots between rows.[15,16] Other approaches have combined LiDAR measurements with particle fil-tering for orchard in-row navigation and with online mapping and localization for navi-gation without prior field mapping.[17,18] More recently, 3D LiDAR has enabled over-canopy crop-row detection and navigation without GNSS-based global localization.[19] For under-canopy operation, CropNav uses LiDAR-based row perception when GNSS reliability degrades and integrates this capability with GNSS-based guidance out-side the rows.[20] However, geometric measurements alone can make it difficult to dis-tinguish crop stalks from hanging leaves and weeds, while missing plants and discon-tinuous rows can reduce navigation accuracy.[17,21]

The robustness of LiDAR-based crop-row navigation is also tied to broader chal-lenges in geometric model fitting, localization, and mapping. Methods such as ener-gy-based multi-model fitting, scan matching, LiDAR odometry, loop closure, and SLAM provide useful tools for extracting structure and reducing drift in repetitive outdoor en-vironments.[22-39] Recent studies on Gaussian splatting, visual-LiDAR SLAM, and sim-ulation-based agricultural mapping further highlight the importance of combining geo-metric consistency with agricultural scene structure under occlusion, repeated rows, and seasonal variation.[30-35]

LiDAR-based crop-row navigation methods commonly rely on the geometric struc-ture of planted rows to estimate the robot's position within the inter-row corridor.[15-20] This makes LiDAR attractive for agricultural navigation because it provides direct range measurements and is less dependent on visual texture or illumination than camera-only approaches. However, the practical performance of LiDAR-based row navigation de-pends strongly on the quality, density, and continuity of the measured point structure.

A key limitation of existing LiDAR-based approaches is their dependence on suffi-ciently clear and continuous row geometry. In real agricultural fields, LiDAR scans may contain missing plant sections, row gaps, weeds, soil returns, tree trunks or branches, and isolated outlier points. These factors distort the apparent row structure and can re-duce the reliability of line extraction, row-center estimation, and heading estimation. Even robust line-fitting and model-fitting methods may become unstable when the number of valid row points is small or when non-row objects dominate the scan.[22-24] Similar difficulties are also observed in orchard environments, where repetitive row geometry, occlusions, vegetation motion, and seasonal changes complicate localization and mapping.[30-34]

Therefore, the main technical problem addressed in this study is robust extraction of crop-row geometry from noisy

and incomplete LiDAR measurements. The problem is not limited to detecting row lines; the detected geometry must also be transformed into navigation-relevant variables, including the inter-row centerline, lateral deviation, and the robot's heading error. In this work, the problem is studied using recorded LiDAR se-quences from the CitrusFarm Dataset, enabling evaluation under realistic agricultural field conditions without conducting a new physical field-data collection campaign.[36]

Although LiDAR-based crop-row navigation has been examined in several studies, important gaps remain in the robustness of existing approaches under realistic field con-ditions. Geometry-based methods are attractive because they are interpretable and com-putationally efficient, but they often require sufficiently continuous row points and can become unstable when weeds, missing plants, branches, ground returns, or sparse Li-DAR measurements dominate the scan.[15-20]

Learning-based crop-row perception methods can improve row or plant-structure extraction when suitable training data are available, but fully supervised approaches usually require manually annotated images, point clouds, or crop-specific retraining. This creates a practical gap for agricultural navigation systems that must operate with limited annotation effort while still producing reliable navigation variables from noisy LiDAR measurements.

Another gap concerns the link between point-level perception quality and naviga-tion-ready behavior. Existing methods may detect row-like structures, but they do not always explicitly determine whether a scan is reliable enough for control. They also do not always specify how the centerline should be estimated when only one row boundary is visible. In addition, the stabilization of lateral and heading errors across consecutive scans is not always explicitly addressed. These issues are critical in orchards and row-crop fields where row geometry can be incomplete or temporarily occluded.

Thus, the research gap addressed in this study is the lack of a reproducible hybrid LiDAR crop-row navigation framework for operation under imperfect field conditions. Such a framework should combine learning-assisted row-point filtering, robust geometric row-line extraction, scan-level reliability assessment, one-row fallback centerline estimation, and temporal stabilization of navigation errors.

The scientific novelty of this study lies in integrating GroveNet, a lightweight clus-ter-level neural network, with reliability-aware geometric row estimation in a unified LiDAR perception pipeline. Through row-boundary reliability checks, one-row fallback, and temporal stabilization, the pipeline reduces fluctuations in centerline and naviga-tion-error estimates under sparse and incomplete canopy conditions.

The main scientific contributions of this study are as follows.

First, an integrated, reliability-aware LiDAR perception pipeline is developed to es-timate the inter-row centerline, lateral deviation, and heading error from sparse LiDAR scans.

Second, a lightweight neural network, GroveNet, is developed to identify crop-row vegetation clusters in LiDAR point clouds and filter out non-row clusters before row-line fitting.

Third, a validation method is developed to assess centerline estimation accuracy using RTK-GNSS robot positions and mapped crop-row boundaries.

The remainder of this article is organized as follows. Section 2 describes the dataset, the proposed LiDAR perception pipeline, and the evaluation methodology. Section 3 presents the experimental results, including comparisons with baseline methods. Section 4 discusses the findings, practical implications, and limitations. Section 5 summarizes the main conclusions and directions for future work.

2. Materials and methods

This section presents the proposed hybrid LiDAR crop-row navigation pipeline and the methodology used to evaluate it. Each LiDAR scan is processed through a navigation-oriented pipeline. The pipeline includes region-of-interest selection, geometric preprocessing, heuristically labeled cluster-level row classification, and robust row-line extraction. It also includes scan-level reliability assessment, inter-row centerline estimation, one-row fallback handling, temporal stabilization of lateral and heading errors, and offline command-generation analysis. The proposed algorithm is evaluated using LiDAR sequences collected in citrus orchards as part of the CitrusFarm Dataset, together with the georeferencing procedure used to construct an independent ground-truth centerline. The section also defines the baseline methods and evaluation metrics used to assess row-detection reliability, navigation-error stability, robustness under degraded field conditions, learning-assisted filtering benefit, and real-time feasibility.

2.1. Proposed Hybrid LiDAR-Based Navigation Pipeline

The proposed hybrid LiDAR-based navigation pipeline is implemented as a scan-by-scan processing procedure applied to recorded CitrusFarm LiDAR sequences. The pipeline converts each raw LiDAR point cloud into navigation-ready variables. These variables include row-point probability estimates, left and right row-boundary estimates, an inter-row centerline, lateral deviation, and heading error. The pipeline also provides a scan reliability flag, a fallback-usage indicator, and a temporally stabilized offline steering correction. The overall structure of the pipeline is shown in Figure 1.

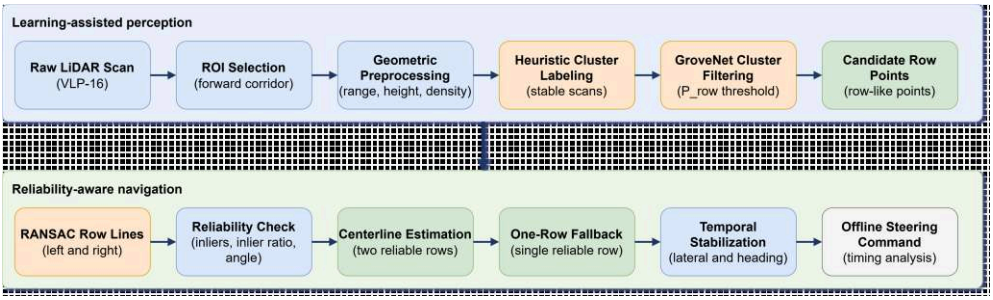


Fig. 1: Hybrid LiDAR crop-row navigation pipeline with learning-assisted point filtering and reliability-aware centerline estimation.

First, each LiDAR scan is processed in the sensor coordinate frame. A radial crop retains points within 0.5–10 m of the sensor, and a rear-azimuth sector of $180° \pm 15°$ is excluded to remove returns from the operator walking behind the robot.

Second, local ground height is estimated to select points within the canopy height band. Statistical outlier removal is then applied, followed by DBSCAN clustering to form the candidate clusters supplied to GroveNet. These preprocessing operations are described in Sections 2.3 and 2.4.

Third, the point clusters produced by geometric preprocessing are passed through a lightweight cluster-level classification module (GroveNet), trained using heuristically labeled clusters. The classifier assigns each cluster a single row-probability score. Only clusters with sufficient row likelihood are retained, and all their points proceed as candidate row-structure points for geometric line extraction.

Fourth, points from GroveNet-accepted clusters are cropped to the rectangular forward-corridor ROI, processed by radius-based density filtering, and voxel-downsampled using the parameters in Table 1. The remaining points are divided into left and right candidate sets according to their lateral coordinates. RANSAC is applied separately to each set to estimate the row boundaries. Each fitted line is then evaluated against the inlier-count, inlier-ratio, and orientation criteria, and unreliable lines are rejected before centerline estimation.

Fifth, the inter-row centerline, lateral deviation, and heading error are estimated from the accepted row lines. If both row lines are reliable, the centerline is computed from the left and right row boundaries. If only one row line is reliable, a one-row fallback centerline is estimated using the expected row spacing. If neither row line is reliable, the scan is marked invalid and the last temporally stabilized estimate is carried forward instead of generating a new, uncertain value. The lateral and heading errors are temporally stabilized using exponential smoothing to reduce scan-to-scan fluctuations.

2.2. Dataset and Georeferenced Experimental Setup

his study evaluates the proposed navigation pipeline offline on recorded sequences from the publicly available CitrusFarm Dataset (University of California, Riverside). The data were collected by a Clearpath Jackal wheeled robot driving between rows of citrus trees and comprise seven sequences, with nine sensing modalities, 1.7 h of operation, 7.5 km of travel, and about 1.3 TB of data.[36]

A subset was selected from the full CitrusFarm Dataset and is referred to throughout this article as field block 13D. The row-line and navigation-error evaluations use 2991 straight-driving LiDAR scans from this subset.

The CitrusFarm data were used not only as a source of LiDAR scans for the navigation algorithm, but also as the basis for its quantitative validation. Since the proposed algorithm estimates row structure in the local sensor coordinate frame, its results must be checked against an independent representation of the robot position and row geometry in a common absolute coordinate system. For this purpose, the LiDAR scans were associated with RTK-GNSS, GNSS, and IMU measurements. These measurements were used to determine the robot pose for each scan and reconstruct the reference row geometry independently of the proposed method. The resulting reference was then used to evaluate the lateral and heading errors of the navigation output.

To construct a common absolute coordinate system, the fixed geodetic datum $(\varphi_0, \lambda_0, h_0)$ was selected as the origin, corresponding to the first RTK-fixed NavSatFix sample of sequence 01 ($\varphi_0 = 33.9634°$, $\lambda_0 = -117.3467°$, $h_0 = 267.87\ m$). The geodetic coordinates were then converted to local metres with a fixed scale $S = 111{,}320$ m per degree:

$$E(\varphi, \lambda) = (\lambda - \lambda_0) \cdot S \cdot \cos \varphi_0, \quad N(\varphi, \lambda) = (\varphi - \varphi_0) \cdot S \tag{1}$$

The RTK trajectory was recorded in its own local coordinate frame and therefore had to be aligned with the ENU frame. The required rigid two-dimensional transform was recovered by least-squares (Kabsch) fitting against the GNSS fixes. The RTK trajectory is linearly interpolated to each GNSS-fix timestamp, giving correspondences (a_i, b_i), where a_i is the interpolated RTK position and $b_i = (E, N)$ is the corresponding GNSS fix. The rotation $R_2 \in \mathrm{SO}(2)$ and translation t_2 that minimize $\Sigma_i \| R_2 a_i + t_2 - b_i \|^2$ are obtained from the singular value decomposition of the cross-covariance of the centered point sets, with a determinant constraint that rules out reflections. Applying (R_2, t_2) to every RTK sample p_k gives the robot's horizontal position in the ENU frame, $r_k = R_2 p_k + t_2$. The mean alignment residual is 4–6 cm and is stored as a georeferencing-quality indicator. The GNSS vertical channel is the noisiest, and the field blocks are nearly flat, so altitude is pinned to a constant $\overline{z}$ — the mean NavSatFix altitude over the evaluated data.

For heading estimation, the CitrusFarm RTK receiver uses a single antenna and therefore does not directly resolve heading, and the MicroStrain yaw channel is not used because it exhibits large transient errors. Heading ψ is therefore taken from the tangent of the lightly smoothed ENU trajectory, $\psi_k = \mathrm{atan2}(\dot{N}_k, \dot{E}_k)$, measured from east, counter-clockwise, wherever the robot moves forward faster than 0.4 m/s with an angular rate below 8°/s (74–85% of the evaluated samples). Elsewhere — headland turns and slow manoeuvres — heading is obtained by integrating the IMU yaw rate ω_z along the trajectory and linearly re-anchoring the integrated curve to the path tangent at the start and end of each such interval. For each scan, the pipeline records whether its heading was anchored to the path tangent or bridged from the gyro.

Roll φ_r and pitch θ_r are obtained from the gravity-referenced AHRS quaternion q_k. The quaternion is converted to a rotation matrix and combined with the IMU-to-LiDAR rotation $R_{\mathrm{IMU}\to\mathrm{LiDAR}}$ from the dataset's camera–IMU–LiDAR calibration, $R' = R(q_k) \cdot R_{\mathrm{IMU}\to\mathrm{LiDAR}}$, from which two angles are read off: $\varphi_r = \mathrm{atan2}(R'_{32}, R'_{33})$ and $\theta_r = \mathrm{atan2}\left(-R'_{31}, \sqrt{{R'_{32}}^2 + {R'_{33}}^2}\right)$. The yaw component of R' is not used.

For a scan acquired at time t, position (E, N) and the angles (φ_r, θ_r) are linearly interpolated to t from the trajectory data, while ψ is interpolated through its sine and cosine to avoid wraparound. The robot body frame follows REP-103 (x forward, y left, z up); the dataset's calibration gives a zero rotation between the LiDAR frame and the body frame, so the sensor axes coincide with the body axes. The body-to-world rotation is

$$R(t) = R_z(\psi) \cdot R_y(\theta_r) \cdot R_x(\varphi_r) \tag{2}$$

The LiDAR origin in the ENU frame is

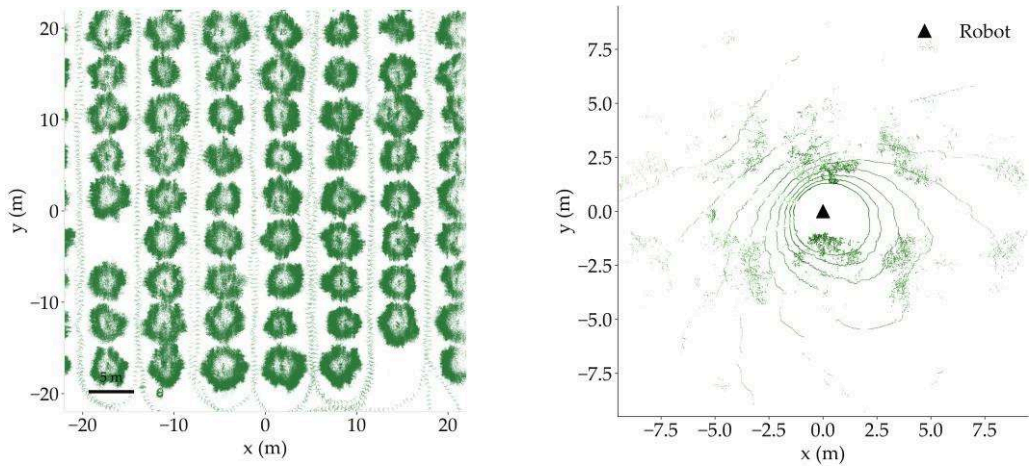
$$o(t) = [E, N, \overline{z}]^{\mathrm{T}} + R(t) \cdot \ell \tag{3}$$

where $\ell = [0.22, 0.00, -0.15]$ m is the GPS-antenna-to-LiDAR offset expressed in the body frame. Each point p^{s} of a scan, given in the sensor frame, is transformed to global coordinates as

$$p^{\mathrm{w}} = R(t) \cdot p^{\mathrm{s}} + o(t) \tag{4}$$

This is applied to every point of every scan; the sensor-frame coordinates , intensity, and ring are left unchanged, and the ENU coordinates are appended. Beyond the LiDAR-to-body rotation given above, the antenna-to-LiDAR lever arm and the IMU-to-LiDAR rotation are the only extrinsic parameters used in this transformation. These parameters are applied here to the georeferenced pose and points, while the navigation algorithm itself continues to operate in the sensor coordinate frame.$(x, y, z)(X, Y, Z)\ell R_{\mathrm{IMU}\to\mathrm{LiDAR}}$

The georeferencing output is stored in HDF5 format, holding about 1.2 billion points and 47,700 scans in total; each point carries both its original sensor-frame values (x, y, z, intensity, ring) and its ENU position (X, Y, Z). Each scan additionally stores its timestamp, its robot pose (position and orientation quaternion), and four per-scan flags: forward speed, yaw-rate, RTK-fixed status, and the heading-anchored indicator. All downstream stages — the navigation pipeline, the baseline methods, the generation of GroveNet training clusters, and the ground-truth construction — read these files directly, without re-decoding the rosbags, so that every method operates on identical inputs. Evaluation frames are then selected per scan from the stored flags: a scan is retained when RTK is fixed, forward speed exceeds 0.5 m/s, and the yaw-rate magnitude is below 4°/s. This admits in-row straight-line driving and excludes headland turns, stops, and RTK dropouts.



(**a**) (**b**)

Fig. 2: Georeferenced canopy mapping used for independent ground-truth construction: **(a)** top-down canopy point-cloud map accumulated in the ENU frame, showing citrus tree rows; **(b)** a single LiDAR scan in the local sensor frame, illustrating the VLP-16's concentric ring pattern within the mapped canopy area.

The independent lateral-error ground truth is derived from the georeferenced canopy. Canopy points (0.4–3.0 m above the locally estimated ground, within 15 m of the sensor, RTK-fixed scans only) are accumulated into a top-down map in the ENU frame (Figure 2a). The crop-row structure is then extracted from this canopy map and represented by robust row lines in the same coordinate system. The true inter-row centerline is the mid-line of the two row boundaries that bound the robot, and a scan's true lateral error is the signed offset of its RTK pose from that centerline, expressed in the pipeline's sign convention. Because this reference is built from the mapped tree rows and the RTK pose — not from the LiDAR pipeline under test — it provides a pipeline-independent check of the reported navigation errors that is used in Section 3.3.

2.3. LiDAR Point Preprocessing and Outlier Filtering

In this study, LiDAR point preprocessing serves as the first filtering stage before learning-assisted cluster classification and crop-row line extraction. The input is a raw Velodyne VLP-16 point cloud from a selected CitrusFarm sequence (Figure 2b), and the output is a geometrically cleaned point set that is passed to the GroveNet cluster classifier described in Section 2.4. Preprocessing is performed independently for each scan to support per-scan timing analysis.

The first operation is coordinate framing. Each LiDAR scan is processed in the sensor's own coordinate frame: the robot's forward direction is treated as the longitudinal x-axis, the left-right direction as the lateral y-axis, and the vertical direction as the z-axis.

After coordinate framing, a radial region of interest keeps points between 0.5 m and 10 m from the sensor, and a fixed rear-azimuth sector (180° ± 15°) is excluded to remove an operator walking behind the robot, identified as a near-constant, person-shaped return in the raw data (Section 2.4). This ROI removes distant trees, background objects, and the trailing operator so that the following stages focus on nearby row-vegetation candidates. The rectangular forward/lateral ROI in Table 1 is applied later, after cluster classification, to the row-candidate points handed to RANSAC (Section 2.5).

The next operation is range and height filtering. Points with invalid range values, extremely short distances, and physically unrealistic coordinates are removed. For 3D point clouds, a height interval is used to reduce ground returns and high-canopy or branch points that do not represent the row boundary used for inter-row guidance.

To reduce isolated outliers and sparse measurements, statistical outlier removal is applied. For each point, the mean distance to its 12 nearest neighbors is compared with the global mean and standard deviation across the scan. Points more than 1.5 standard deviations from the mean are removed. The retained points are then grouped into candidate clusters using DBSCAN. A second, radius-based density filter (Table 1: 0.30 m radius, minimum 3 neighbors) is applied to the row-candidate points immediately before RANSAC line fitting (Section 2.5), reducing the influence of isolated weed returns, leaf-edge points, and occasional sensor reflections that remain after GroveNet filtering.

The height-band-filtered, outlier-removed, and clustered point set described above is passed directly to the GroveNet cluster classifier (Section 2.4) without further cropping or downsampling. Only after classification are the points of GroveNet-accepted clusters passed through the rectangular ROI, the radius-based density filter, and a voxel-grid downsampling step (Table 1) to reduce computational cost while preserving row geometry for RANSAC line fitting. The default implementation parameters used to make the pipeline reproducible are summarized in Table 1.

Table 1: Default parameters of the proposed hybrid LiDAR navigation pipeline.

Stage	Parameter	Default value
ROI selection (forward)	Forward range x	0.7 to 7.0 m
ROI selection (lateral)	Lateral range y	-3.0 to 3.0 m
Height filtering	Height interval z	0.15-2.50 m
Density filtering (radius)	Neighbor radius	0.30 m
Density filtering (min. points)	Minimum neighbors	3 points
Downsampling	Voxel size	0.10 m
RANSAC (distance)	Distance threshold	0.20 m
RANSAC (iterations)	Maximum iterations	100
Reliability check (inliers)	Minimum inliers	25 points
Reliability check (ratio)	Minimum inlier ratio	0.35
Reliability check (orientation)	Orientation tolerance	±25 deg
GroveNet classifier (input)	Input features	512×4
GroveNet classifier (threshold)	Row-probability threshold	0.5
Temporal stabilization	Smoothing factor alpha	0.1
Geometric preprocessing (clustering)	DBSCAN neighborhood radius / minimum points	0.35 m / 8 points

2.4. Cluster-Level Row Classification (GroveNet)

The learning-assisted filtering stage is placed between geometric preprocessing and RANSAC-based row-line extraction. Geometric preprocessing (Section 2.3) groups the cleaned point cloud into candidate clusters using

DBSCAN (neighborhood radius 0.35 m, minimum 8 points per cluster), applied to points within the locally estimated canopy height band. Each cluster is then classified as row vegetation or non-row vegetation by GroveNet, and only points belonging to accepted clusters are retained for line fitting.

Because the CitrusFarm Dataset provides no manual cluster-level row labels, training samples and initial labels are generated by an offline auto-labeling pipeline applied directly to the raw LiDAR frames of each sequence, independent of the online RANSAC row-line estimate. The automatically generated labels are then manually reviewed. Each LiDAR frame is preprocessed using radial cropping and rear-sector exclusion, as described in Section 2.3. Local ground height is estimated as the 5th percentile of z within 0.4 m × 0.4 m XY grid cells to account for terrain variation. Canopy height filtering, statistical outlier removal, and DBSCAN clustering are then applied with the same preprocessing settings as the online pipeline (Section 2.3 and Table 1).

Each DBSCAN cluster is labeled using a geometric shape heuristic (is_bush_cluster, Table 1) rather than a distance-based comparison to fitted row lines. A cluster is labeled row-vegetation when it satisfies four geometric criteria. It must contain at least 30 points and have a vertical extent of at least 0.4 m, which excludes flat ground strips. Its horizontal footprint in both x and y must not exceed 4.5 m, which excludes merged multi-row blobs. Finally, its height-to-width aspect ratio must not exceed 3.0, which helps exclude narrow, person- or pole-shaped objects outside the rear sector. Clusters failing any of these criteria are labeled non-row. This rule identifies citrus-canopy-shaped vegetation in general rather than proximity to a specific row line; within the LiDAR's forward-looking region of interest, such clusters are assumed to predominantly correspond to row-relevant vegetation. Non-row training examples are additionally drawn from DBSCAN noise points (isolated returns) and the full set of height-band-excluded points (ground layer and above-canopy). The excluded points are tiled into non-overlapping 2.5 m × 2.5 m grid cells containing at least 5 points each. This ensures that every excluded point is represented in a training sample.

Each labeled cluster is represented as a fixed-size point set of 512 points with four channels per point (x, y, z, LiDAR intensity), with x, y, z centered on the cluster's own centroid. Clusters with more than 512 points are reduced via farthest-point sampling, which preserves spatial coverage; clusters with fewer points are padded by randomly repeating existing points.

GroveNet processes each cluster using a residual point-based encoder with three residual PointBlocks containing 48, 96, and 192 channels. Global max- and mean-pooling are then applied over the point dimension, and the resulting descriptors are concatenated. Two fully connected layers with 64 and 32 units lead to a sigmoid output neuron that predicts the cluster's row probability P_{row}. The resulting model has approximately 75K parameters (Table 2).

The model is trained as a binary classifier using the heuristic row/non-row labels, binary cross-entropy loss, and the Adam optimizer, following the same data split, sampling strategy, and training schedule as the baseline models compared in Section 3.1. To reduce data leakage, labeled frames are split by sequence or contiguous scan interval rather than mixing individual clusters from the same frame into both training and validation sets.

During inference, the trained classifier computes P_{row} for each candidate cluster produced by geometric preprocessing. Clusters with $P_{\mathrm{row}} \geq 0.5$ are retained, and all of their points proceed to RANSAC; clusters below this threshold are discarded entirely. This filtering is expected to increase the RANSAC inlier ratio, reduce false row-line fitting caused by weeds, branches, ground returns, or isolated outliers, and improve the stability of the subsequent centerline and navigation-error estimates.

2.5. Crop-Row Line Extraction

The crop-row line extraction stage converts the filtered LiDAR point set from Sections 2.3 and 2.4 into two geometric models: the left and right crop-row lines. This stage is conceptually related to prior LiDAR-only crop navigation work, in which a PEARL/Ruby-based line-finding algorithm was used to extract crop-row models before fuzzy control.[16] n this study, learning-assisted RANSAC line fitting was selected as the row-extraction method because it provides a reproducible and robust geometric approach for noisy CitrusFarm LiDAR scans.

The row-probability-filtered point cloud is first separated into left-row and right-row candidate sets in the robot-centered coordinate frame. After applying the lateral ROI limits defined in Table 1, points with positive lateral coordinates are assigned to one row side, and points with negative lateral coordinates are assigned to the opposite row side. Points near the robot centerline can be ignored if they are unlikely to belong to row boundaries.

For each candidate set, RANSAC estimates a two-dimensional line model in the ground-plane projection of the point cloud. In each iteration, two points are sampled to define a candidate line. The perpendicular distance from all candidate points to this line is computed, and points within the 0.20 m distance threshold are treated as inliers. After 100 iterations, the model with the largest inlier support is selected as the preliminary row boundary.

After RANSAC selection, the final row line is refined using all inlier points via least-squares fitting. This refinement reduces random variation from the initial RANSAC sampling and yields a stable estimate of row direction and position. The same procedure is applied independently to the left and right candidate sets.

Each extracted line is then evaluated against scan-level reliability criteria. A row line is accepted only if the number of inliers, inlier ratio, and line orientation meet the thresholds in Table 1. If one side has too few reliable points due to missing plants, partial occlusion, or sparse LiDAR returns, that row line is marked as unreliable for the current scan.

The output of this stage includes the left and right row-line parameters, their corresponding inlier point sets, and quality indicators such as inlier count, inlier ratio, and reliability status. These outputs are passed to the next stage, where the inter-row centerline, lateral deviation, and heading error are computed for navigation command generation.

2.6. Row-Center Estimation and Navigation Error Computation

The row-center estimation stage uses the left and right crop-row lines extracted in Section 2.5 to compute the desired navigation path between the rows. The input consists of the accepted left row line, the accepted right row line, and their reliability indicators. The output consists of the inter-row centerline, the robot's lateral deviation, and the robot's heading error.

In the robot-centered coordinate frame, each row boundary is represented in slope-intercept form as $y = ax + b$, where a is the row-line slope and b is the lateral intercept at $x = 0$. This form is adequate here because, within the forward-looking ROI (Table 1), row lines remain roughly aligned with the forward direction rather than near-vertical in the (x, y) scan plane. The left and right row lines are denoted as L_{left} and L_{right}.

When both row lines are reliable, the inter-row centerline is computed by sampling forward positions x_i within the ROI, estimating the corresponding lateral row positions $y_{\text{left}}(x_i)$ and $y_{\text{right}}(x_i)$, and computing:

$$y_{\text{center}}(x_i) = \left(y_{\text{left}}(x_i) + y_{\text{right}}(x_i)\right) / 2 \tag{4}$$

A straight centerline L_{center} is then fitted through the center points. This centerline represents the desired path for the current scan.

If only one crop-row line is reliable, the centerline is estimated using a smoothed memory of the inter-row corridor. This memory is constructed from recent scans in which both row lines were reliable. It uses an exponential moving average ($\alpha = 0.1$) of the row spacing, corridor slope, and each side's perpendicular distance from the robot. The one-row centerline is obtained by shifting the smoothed corridor center according to the visible row. The shift is set to a fraction (0.3) of the change in the visible row's perpendicular position since both rows were last reliable. It is clamped to ±0.35 m per scan, while the corridor slope is taken directly from the smoothed memory. This damped, memory-anchored update is intended to reduce sensitivity to changes in the visible row-line estimate. If neither row line satisfies the reliability criteria, or no such memory yet exists, the scan is marked invalid and excluded from navigation-error computation.

The lateral deviation e_y is computed as the signed perpendicular distance from the robot reference point to L_{center}. Since the robot-centered frame places the robot reference point at the origin, the sign of e_y indicates whether the robot is located to the left or right of the estimated inter-row path. The heading error e_θ is computed as the difference between the orientation of L_{center} and the robot forward direction; in the robot-centered frame, the robot heading angle is zero.

For each processed LiDAR scan, the algorithm stores the centerline parameters, e_y, e_θ, row-line reliability flags, inlier counts, inlier ratios, row spacing estimate, and scan-validity status. To reduce scan-to-scan fluctuations, the lateral deviation and heading error are temporally stabilized using an exponential smoothing update:

$$\tilde{e}(t) = \alpha \cdot e(t) + (1 - \alpha) \cdot \tilde{e}(t - 1) \tag{5}$$

where controls the balance between current LiDAR evidence and the previous stable navigation estimate. These values provide the basis for navigation command generation and for the quantitative evaluation reported in Section 3.

2.7. Baseline Methods for Comparison

The full proposed hybrid pipeline is compared with two baseline methods: raw RANSAC and geometric filtering plus RANSAC. All three methods are evaluated on the same 2991 straight-driving LiDAR scans from field block 13D. The comparison assesses row-line acceptance, lateral-error variability, and computational cost.

The first baseline is raw RANSAC line fitting. In this configuration, line extraction is applied directly after basic region-of-interest selection, without geometric filtering, learned row-point filtering, scan-level reliability assessment, one-row fallback, or temporal stabilization.

The second baseline is geometric filtering plus RANSAC. This configuration applies range, height, density, and downsampling filters before RANSAC, but does not use the learned cluster classifier. It measures the contribution of classical LiDAR preprocessing.

The final method is the full proposed hybrid pipeline, which combines geometric preprocessing, heuristically labeled cluster-level row classification, RANSAC row-line extraction, scan-level reliability assessment, one-row fallback centerline estimation, temporal stabilization, and offline command generation.

2.8. Evaluation Metrics

The evaluation metrics are defined to measure the proposed method at three levels: learning-based row-candidate classification, geometric row-line estimation, and computational feasibility. The experiments are performed on recorded CitrusFarm LiDAR sequences rather than closed-loop robot trials. Therefore, the evaluation focuses on row-detection reliability, line-fitting quality, centerline availability and stability, robustness under different field conditions, and processing time. Physical navigation success is not evaluated.

Row-candidate classification performance is evaluated using accuracy, precision, recall, F1-score, and validation loss. These metrics are computed on held-out heuristically labeled LiDAR clusters and are used to compare GroveNet with the baseline learning models. The classifier evaluation is reported separately from the row-line evaluation because high classification performance does not automatically guarantee stable row-line estimation.

All row-line and centerline metrics are computed scan by scan and then aggregated over the selected CitrusFarm LiDAR scan intervals. For each scan and evaluated method, the pipeline records the number of candidate row points and RANSAC inliers. It also records the fitted left and right row-line parameters, row-line acceptance flags, point-to-line residuals, centerline validity, and fallback usage. The processing time of the main pipeline stages is recorded separately.

Let N denote the total number of evaluated LiDAR scans. For scan t, L_t and R_t are binary indicators equal to 1 when the left and right row lines satisfy the reliability criteria defined in Section 2.5, and equal to 0 otherwise. The left-row detection rate and the right-row detection rate are computed as:

$$(\Sigma_t L_t) \,/\, N \times 100\%, \qquad (\Sigma_t R_t) \,/\, N \times 100\% \tag{6}$$

The both-row detection rate is computed as the percentage of scans for which L_t = 1 and R_t = 1. The at-least-one-row detection rate is computed as the percentage of scans for which L_t = 1 or R_t = 1. The both-row detection rate indicates how often the inter-row corridor can be estimated directly from two reliable row boundaries, while the at-least-one-row detection rate indicates how often at least one reliable crop-row boundary remains available for row-structure estimation. Of these four rates, Table 6 reports the at-least-one-row detection rate as "Scans with ≥1 accepted row line (%)"; the left-row, right-row, and both-row rates are defined for completeness but are not separately tabulated.

For each accepted row line, the RANSAC inlier ratio is computed as:

$$N_{\text{inlier}} \,/\, N_{\text{candidate}} \tag{7}$$

where N_{inlier} is the number of inlier points selected by RANSAC and $N_{\text{candidate}}$ is the number of candidate row points supplied to the line-fitting stage. The fitted row line is represented in slope-intercept form as $y = ax + b$ (Section 2.6). For an inlier point $p_i = (x_i, y_i)$, the perpendicular distance to the fitted line is computed as:

$$d_i = |a \cdot x_i - y_i + b| \,/\, \sqrt{a^2 + 1} \tag{8}$$

The mean line residual and residual standard deviation are computed from these distances over all accepted row lines. A higher inlier ratio and a lower residual indicate that the candidate points are more consistent with a row-like linear structure.

Centerline availability is computed as:

$$N_{\text{centerline}} / N \times 100\% \tag{9}$$

where $N_{\text{centerline}}$ is the number of scans in which an inter-row centerline can be estimated either from two reliable row lines or from the one-row fallback procedure defined in Section 2.6. Fallback usage is computed as:

$$N_{\text{fallback}} / N_{\text{centerline}} \times 100\% \tag{10}$$

where N_{fallback} is the number of valid centerline estimates generated from only one reliable row boundary.

The pipeline's self-reported lateral and heading errors are additionally cross-checked against the independent ground-truth centerline of Section 2.2. The mean absolute lateral error is the average absolute difference between the pipeline's stabilized lateral-deviation estimate and the ground-truth lateral position, over all scans with a valid ground-truth centerline. The signed bias is the mean signed difference between the two (pipeline estimate minus ground truth), indicating systematic offset rather than random error. The correlation r between the two series indicates how closely the pipeline's estimate tracks the ground truth's own scan-to-scan variation. The mean and standard deviation of the heading error are computed directly from the pipeline's stabilized heading-error estimate.

Robustness is reflected in how the pipeline behaves when row structure is only partially visible or absent, rather than through a breakdown into discrete visibility categories. The one-row fallback usage rate (Table 4) indicates how often the centerline is reconstructed from a single reliable row line, while scans in which neither row line is reliable rely on the carried-forward temporally stabilized estimate (Section 2.6) instead of a fresh, uncertain one. This shows whether the method avoids generating unstable navigation variables as row visibility degrades.

Computational performance is evaluated by measuring the processing time per scan for ROI selection, geometric preprocessing, GroveNet inference, RANSAC row-line fitting, reliability checking, centerline estimation, and total pipeline execution. For each stage, the mean processing time is reported. The method is considered computationally feasible when the average total processing time per scan remains below the nominal LiDAR scan period.

All row-line, centerline, robustness, and timing metrics are computed separately for the geometry-only method, the GroveNet-assisted method, and the full hybrid method using the same selected scan intervals. This ensures that the comparison reflects the contribution of learned filtering and reliability-aware row estimation rather than differences in data selection.

3. Results

This section reports measured results from the proposed hybrid LiDAR crop-row navigation pipeline, evaluated on real LiDAR sequences from the CitrusFarm Dataset as described in Section 2. Table 2's classification results and Figures 3-5 were obtained from the trained GroveNet model and its baselines; Tables 4-6's navigation-level results were obtained from the RANSAC-based row-line extraction and control-variable pipeline described in Sections 2.5-2.6, run on 2991 straight-driving LiDAR scans from field block 13D and cross-checked against an independent RTK- and canopy-map-based ground-truth centerline; Table 5's timing was additionally measured on the target deployment device.

The results are organized around six aspects: row-point classification performance, the effect of learned filtering on row-line estimation quality, navigation-error stability and robustness, computational performance, comparison with baseline methods, and an RTK-based cross-check of the navigation-error estimates. This structure allows the proposed method to be evaluated not only as a row-detection algorithm but also as a complete navigation pipeline that produces centerline, lateral error, and heading error, with an independent check to determine whether the self-reported errors reflect the robot's true position.

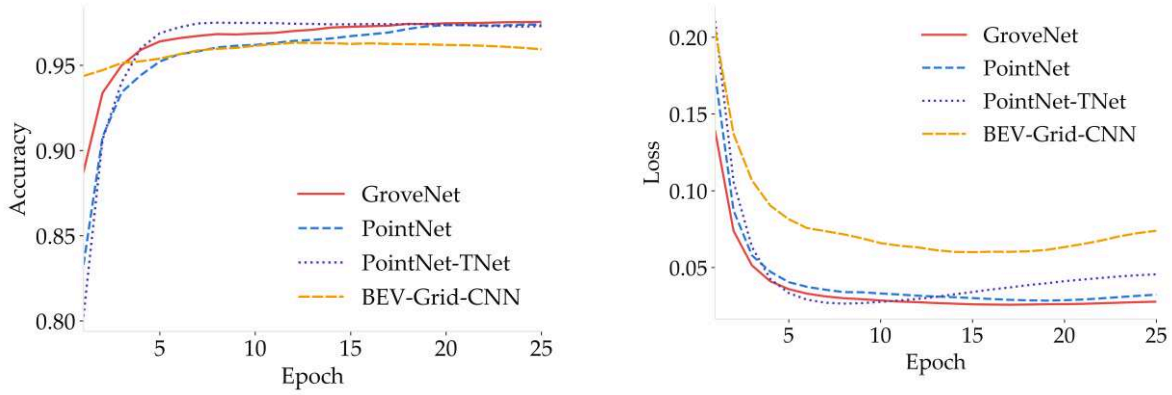
3.1. GroveNet Training and Row-Point Classification Performance

The first experiment evaluates the learning-assisted filtering component of the proposed hybrid navigation pipeline. The purpose of this experiment is to determine whether the proposed GroveNet model can distinguish row-vegetation clusters from non-row or hard-negative LiDAR clusters before geometric row-line extraction. This stage is important

because false row candidates, isolated vegetation returns, and non-row objects can reduce the stability of RANSAC-based row fitting in the subsequent navigation stage.

GroveNet was trained and evaluated using the same data split, sampling strategy, optimizer configuration, and training schedule as the baseline models. Each input sample was represented as a fixed-size LiDAR point cluster with 512 points and four channels, consisting of x, y, z, and intensity values. The model outputs a single probability score indicating whether the input cluster belongs to row-related vegetation. The comparison included PointNet, PointNet-TNet, and BEV-Grid-CNN as baseline learning models.

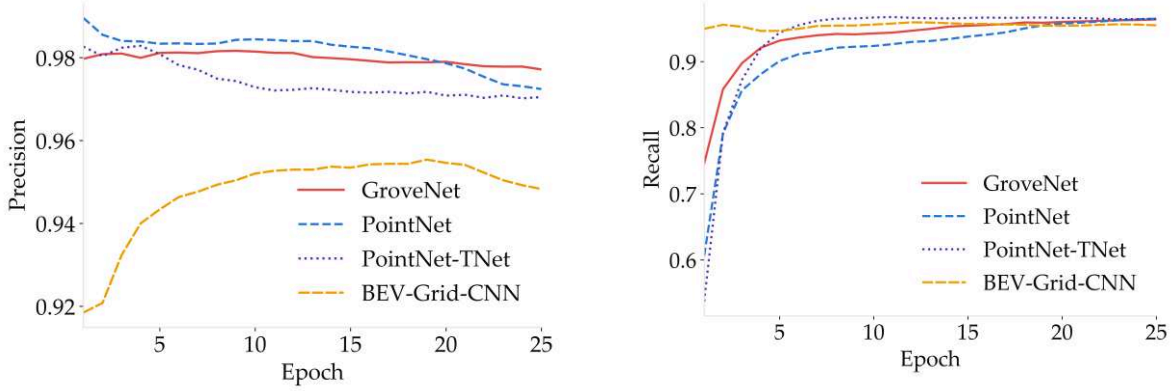
Figure 3 shows the validation accuracy and validation loss curves for GroveNet and the baseline models. GroveNet reached a high validation accuracy and maintained a low validation loss during training, indicating stable convergence. PointNet also achieved competitive accuracy, while BEV-Grid-CNN showed lower classification performance. PointNet-TNet converged rapidly in the early epochs, but its validation loss increased after the initial training stage, suggesting reduced stability compared with GroveNet.



(**a**) (**b**)

Fig. 3: Validation accuracy and loss curves of GroveNet and baseline models during training: (**a**) validation accuracy; (**b**) validation loss.

Figure 4 presents the validation precision and recall curves. The results show that GroveNet provides a strong balance between precision and recall. This balance is important for navigation because high recall helps retain true row-vegetation clusters, while high precision reduces the number of non-row clusters passed to the geometric line-fitting stage. Among the tested models, GroveNet achieved the best overall F1-score on the held-out test set.



(**a**) (**b**)

Fig. 4: Validation precision and recall of GroveNet and baseline models during training: (**a**) precision; (**b**) recall.

Table 2 summarizes the test-set classification results. GroveNet achieved the highest F1-score of 0.9768, with a precision of 0.9846 and a recall of 0.9691. Compared with PointNet, GroveNet improved the F1-score while requiring only a moderate number of parameters. Although PointNet-TNet reached a comparable F1-score, its parameter count

and training time were much larger, and its validation loss behavior was less stable. Therefore, GroveNet was selected as the learned filtering module for the proposed hybrid LiDAR navigation pipeline.

Table 2: Test-set classification performance of GroveNet and baseline models.

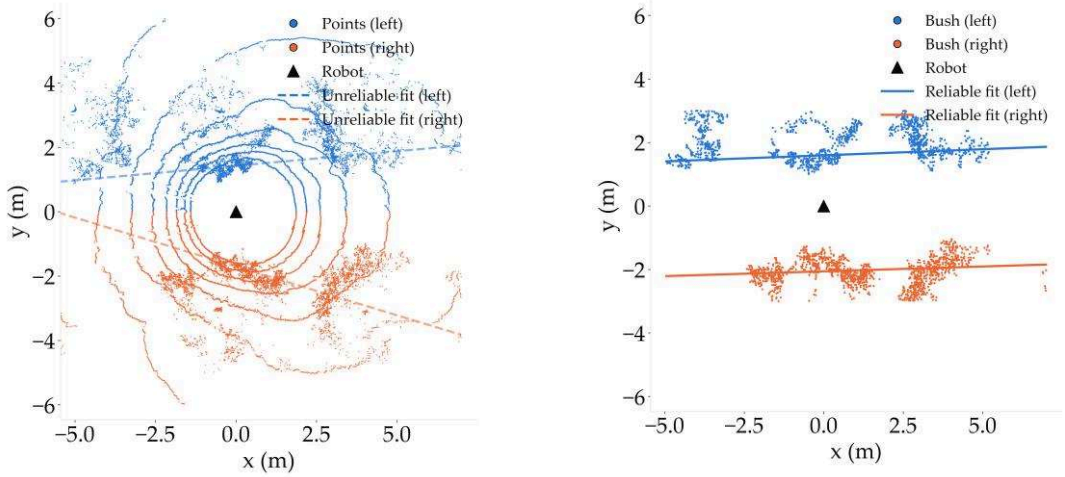
Model	Parameters	Time per epoch	Test F1	Precision	Recall
GroveNet	75K	~7 s	0.9768	0.9846	0.9691
PointNet	443K	~15 s	0.9759	0.9835	0.9685
PointNet-TNet	3.1M	~42 s	0.9748	0.9841	0.9657
BEV-Grid-CNN	6K	~3 s	0.9651	0.9746	0.9557

These results indicate that GroveNet is suitable for use as a pre-filter before row-line extraction. The model improves the quality of the candidate row-structure set while remaining computationally lighter than larger PointNet-TNet-style architectures. In the following subsection, GroveNet is integrated with geometric row-line estimation to evaluate its effect on row-line acceptance and fitting quality.

3.2. Effect of GroveNet Filtering on Row-Line Estimation

To evaluate the effect of the learned filtering stage on row-line estimation, the same representative LiDAR scan was processed with two configurations: a geometry-only row-fitting method and the proposed GroveNet-assisted row-fitting method. In the geometry-only configuration, row lines were estimated directly from the geometrically preprocessed candidate points. In the GroveNet-assisted configuration, the same candidate points were first passed through GroveNet, and points classified as non-row candidates were removed before row-line fitting.

Figure 5 compares the row-line estimation outputs of the two configurations. In the geometry-only result, the fitted row lines are influenced by scattered off-row vegetation returns and isolated candidate points. These points remain in the fitting stage and can affect the estimated row direction and row boundary position. In contrast, the GroveNet-assisted result shows that several non-row candidates are rejected before line fitting. After this filtering step, the retained points are more concentrated around the left and right row structures.



(**a**) (**b**)

Fig. 5: Row-line estimation without and with GroveNet-assisted filtering: (**a**) geometry-only mathematical row fitting; (**b**) GroveNet-assisted row fitting.

The visual comparison shows that GroveNet filtering produces a cleaner candidate set for the subsequent geometric row-line extraction stage. Compared with the geometry-only configuration, the GroveNet-assisted configuration reduces the influence of isolated non-row points and provides row-line estimates that are more consistent with the visible row structure. This result confirms that GroveNet acts as a learned pre-filter rather than replacing the geometric row-estimation stage.

To quantify this effect beyond the single-scan comparison in Figure 5, both configurations were evaluated on all 2991 straight-driving LiDAR scans from field block 13D. The same RANSAC thresholds and scan-level reliability

criteria were used for both configurations (Table 1). The row-line quality metrics defined in Section 2.8 were then aggregated over all accepted lines (Table 3). GroveNet filtering reduces the mean number of candidate points handed to RANSAC from 2841 to 2298 per frame (19.1% fewer). The number of accepted row lines increased from 2554 to 3215 (25.9%), while the mean inlier ratio increased from 0.394 to 0.419. The mean point-to-line residual changed from 9.61 to 9.58 cm, and its standard deviation remained essentially unchanged at 5.80 and 5.79 cm, respectively. These results indicate improved availability of accepted row geometry, with little change in the residual statistics of accepted fits.

Table 3: Comparison of RANSAC row-line fitting with and without GroveNet filtering.

Configuration	Mean inlier ratio	Mean residual (cm)	Residual std (cm)	Mean candidate points
Geometry-only	0.394	9.61	5.80	2841
GroveNet-assisted	0.419	9.58	5.79	2298

3.3. Navigation-Error Stability and Robustness Analysis

The navigation estimates were quantitatively evaluated using the version of the CitrusFarm data prepared in Section 2.2, in which the original LiDAR measurements were preserved and supplemented with absolute coordinates obtained through RTK-GNSS georeferencing. The algorithm estimates row structure from local LiDAR data, while the georeferenced canopy map is used to construct the reference centerline. This reference is constructed separately from the algorithm under evaluation and is not used to correct its output estimates. Using these two coordinate representations of the data enables the local perception results to be checked against the mapped inter-row geometry in a common absolute coordinate system.

Table 4 summarizes the lateral-error comparison against the georeferenced reference centerline, along with statistics on the pipeline's heading-error estimates. The evaluation covers 2991 straight-driving scans from field block 13D, with sample availability for each comparison specified below.

Table 4: Aggregate navigation-error metrics against ground truth.

Metric	Value
Mean lateral error (cm)	13.91
Mean heading error (deg)	-0.08
Heading error std (deg)	2.66
Correlation r (self-estimate vs. ground truth)	0.90
Signed bias (self − ground truth, cm)	-3.85

The proposed method was evaluated using the GroveNet classifier on an NVIDIA Jetson AGX Orin, using the 2991 straight-driving scans from field block 13D selected as described in Section 2.2. A reference centerline was available for 2989 scans; the lateral-error, correlation, and bias metrics were computed for the 2918 scans with both a reference value and a finite algorithm estimate. The mean absolute lateral error was 13.91 cm, and the mean signed difference was -3.85 cm. The correlation between the estimated and reference lateral deviations was 0.90. The algorithm's heading-error estimates had a mean of −0.08° and a standard deviation of 2.66°.

3.4. Computational Performance

Computational performance is evaluated by measuring the processing time required for each stage of the proposed hybrid pipeline. The measured stages include ROI selection, geometric preprocessing, GroveNet cluster classification, RANSAC-based row-line extraction, reliability assessment, row-center estimation, navigation-error stabilization, and control command generation. The total processing time per scan is compared with the expected LiDAR scan period to assess real-time feasibility.

Table 5 reports the measured processing time for each pipeline stage, obtained on the same 2991 LiDAR scans described in Section 3.3, on the target deployment device. The method is considered computationally feasible for real-

time navigation if the average processing time remains below the nominal LiDAR scan period; as discussed below, the current implementation meets this criterion after optimizing the geometric-preprocessing stage.

Table 5: Measured computational performance per LiDAR scan on the target deployment device (NVIDIA Jetson AGX Orin).

Pipeline stage	Mean time (ms)
ROI selection	2.64
Geometric preprocessing	79.97
GroveNet cluster classification	4.83
RANSAC and reliability assessment	12.43
Centerline, stabilization, and command	0.06
Total hybrid pipeline	99.93

Timing was measured on an NVIDIA Jetson AGX Orin in MAXN power mode with jetson_clocks enabled across the same 2991 LiDAR scans. As shown in Table 5, geometric preprocessing accounted for the largest share of the mean total processing time, requiring 79.97 ms per scan (80.0%). GroveNet classification required 4.83 ms (4.8%), while RANSAC-based row-line extraction and reliability assessment required 12.43 ms (12.4%). The complete pipeline had a mean processing time of 99.93 ms, below the 100 ms scan period of the VLP-16 operating at 10 Hz. These measurements support the computational feasibility of the proposed pipeline at the sensor rate under the tested conditions.

3.5. Comparison with Baseline Methods

This subsection compares the proposed hybrid pipeline with two baseline methods defined in Section 2.7: raw RANSAC and geometric filtering + RANSAC. All three methods were applied to the same 2991 straight-driving LiDAR scans from field block 13D. The comparison evaluates the percentage of scans with at least one accepted row line and processing time.

Table 6 summarizes the performance of the three methods. The percentage of scans with at least one accepted row line was calculated using the same reliability criteria defined in Table 1 for all methods. This metric measures the availability of newly accepted row geometry rather than overall navigation success. For the two baselines, the criteria were applied only for evaluation; in the proposed method, they also governed the use of row lines for centerline estimation. Row-line acceptance and lateral-error statistics were obtained on the desktop workstation, with lateral errors evaluated against the independent ground-truth centerline described in Section 2.2. Processing times were measured on the NVIDIA Jetson AGX Orin using the procedure described in Section 3.4.

Table 6: Performance comparison of the proposed hybrid pipeline and baseline methods.

Method	Scans with ≥1 accepted row line (%)	Time per scan (ms)
Raw RANSAC	2.88	22.06
Geometric filtering + RANSAC	70.01	90.04
Full proposed hybrid method	86.09	99.93

Raw RANSAC produced at least one accepted row line in 2.9% of scans, compared with 70.0% for geometric filtering + RANSAC and 86.09% for the proposed hybrid pipeline. Relative to the geometric baseline, the proposed method increased the row-line acceptance rate by 16.08 percentage points, corresponding to a relative increase of approximately 23.0%. This improvement in row-line availability required additional computation: mean processing time increased from 90.04 ms for the geometric baseline to 99.93 ms for the proposed method, while raw RANSAC

required 22.06 ms. Nevertheless, all three methods had mean processing times below the 100 ms scan period of the 10 Hz LiDAR, supporting computational feasibility at the sensor rate on average under the tested conditions.

4. Discussion

Integrating learned cluster filtering with geometric row estimation provides a means of addressing noisy and incomplete LiDAR observations in agricultural navigation. The discussion examines how this combination affects row-line detection and the resulting navigation estimates, considering the balance between geometric consistency, estimation accuracy, and computational cost. It also addresses the limitations of the current evaluation and the requirements for subsequent deployment.

4.1. Interpretation of Navigation Performance

Geometric row estimation depends on whether the observed points provide sufficiently clear evidence of the planted rows. Off-row vegetation and isolated returns can obscure this structure and support competing line hypotheses, making the intended boundaries difficult to identify. Selecting candidate clusters before line fitting offers an additional way to reduce this ambiguity. In the proposed pipeline, GroveNet selects clusters using learned features, while RANSAC estimates row geometry from the retained points. This division of responsibilities combines learned selection with explicit geometric constraints, complementing the outlier handling and model refinement explored in earlier LiDAR-based navigation work.[15]

The comparison in Section 3.2 illustrates how this selection affects geometric fitting. GroveNet reduced the mean candidate count from 2841 to 2298 points per scan, approximately 19.1%, while the number of accepted row lines increased from 2554 to 3215 (25.9%). Thus, the smaller candidate set supported more line estimates that satisfied the same reliability criteria. Among accepted lines, the mean inlier ratio increased from 0.394 to 0.419. The residual standard deviation decreased slightly from 5.80 to 5.79 cm. Similarly, the mean residual changed only slightly, from 9.61 to 9.58 cm (Table 3). These results are consistent with reduced interference from unsuitable candidates: the retained points supported more accepted fits with less residual dispersion. The improvement therefore concerns the consistency and availability of fitted geometry, rather than a substantial reduction in average point-to-line distance.

This increased availability matters because accepted row observations provide opportunities to update the estimated inter-row corridor. The full pipeline produced at least one accepted row line in 86.09% of scans, compared with 70.01% for geometric filtering plus RANSAC and 2.88% for raw RANSAC (Table 6). Relative to the geometric baseline, this represents an increase of 16.08 percentage points, or approximately 23.0%. However, even the full pipeline obtained accepted geometry in more than four-fifths of the evaluated scans. The result therefore demonstrates an improvement under challenging observation conditions while also showing that reliable geometric updates remained intermittent.

A methodological strength of this study is the integration of a separate reference-based validation pathway into the data preparation procedure (Section 2.2). The original sensor-frame LiDAR measurements were preserved for algorithm processing, while synchronized positioning and orientation data enabled the construction of a georeferenced canopy map. The reference centerline was determined midway between the mapped row boundaries, and the signed perpendicular distance from the robot's RTK-derived position to this centerline defined the reference lateral deviation. This reference was constructed separately from the evaluated row-estimation pipeline and was not used to correct its outputs. The resulting comparison provided a traceable quantitative assessment of the navigation estimates against mapped field geometry, as reported in Section 3.3 and Table 4.

The effect on lateral estimation was more limited. Against the reference centerline, the full pipeline had a mean absolute lateral error of 13.91 cm and a signed bias of −3.85 cm (Table 4). The small bias indicates little average directional offset, while the correla-tion of 0.90 indicates a strong positive association between the estimated and reference lateral deviations. However, the mean absolute error shows that individual discrepan-cies remained. Incomplete row observations and temporal processing may help explain this gap between improved fitting and lateral estimation, although their contributions were not isolated experimentally. The demonstrated benefit is therefore more frequent acceptance of row geometry; a substantial improvement in navigation-estimation accu-racy is not established by these results.

4.2. Practical Advantages for Agricultural Robots

For an agricultural robot, row perception must produce updated estimates quickly enough to keep pace with incoming sensor measurements. The processing times reported in Table 5 support this requirement: the complete pipeline required 99.93 ms per scan on average, below the 100 ms period of the 10 Hz LiDAR. These measurements were ob-

tained on an NVIDIA Jetson AGX Orin, an embedded computing platform designed for robotics and autonomous systems. Evaluating the method on such hardware makes the timing results directly relevant to onboard implementation. Under the tested configura-tion, the mean processing time was below the interval between successive acquisitions, supporting computational feasibility on average.

The practicality of adding learned filtering also depends on whether its computational demands remain manageable within the complete pipeline. GroveNet required 4.83 ms per scan, accounting for 4.8% of the total processing time. Its classification performance was close to that of PointNet while using approximately 75K rather than 443K parameters (Table 2). This combination supports the use of a compact classifier to improve candidate selection within a limited processing budget. Geometric preprocessing accounted for the largest share of runtime, approximately 80.0%. Therefore, further reductions in total processing time should primarily focus on point-cloud processing and clustering. The measured cost distribution therefore helps identify where optimization would have the greatest practical value.

Previous studies have demonstrated several ways of connecting LiDAR perception to agricultural navigation. Malavazi et al. investigated geometric line extraction with outlier handling and model refinement, including tests on an agricultural robot.[15] Blok et al. evaluated particle-filter-based navigation in an actual apple orchard, including configurations with missing trees.[17] Their experiments establish that field navigation and incomplete row structure have already been addressed through geometric and probabilistic approaches. Differences in sensors, environments, reference definitions, and closed-loop operation prevent a direct numerical ranking against the present offline results. The contribution of this study is consequently supported by its integration of compact learned cluster filtering, explicit geometric reliability assessment, and centerline estimation evaluated on recorded orchard scans and embedded hardware.

4.3. Limitations and Future Improvements

The current geometric model approximates the local crop corridor using two straight row boundaries, consistent with the straight-driving sequences evaluated in this study. Its applicability to curved rows, headland turns, and row intersections remains to be assessed. Future work could incorporate local curve or spline models for curved rows and evaluate additional strategies for transitions between rows.

Beyond the shape of the corridor, performance also depends on the availability of sufficient row information in the LiDAR measurements. Although learned selection of candidate clusters improves geometric fitting, as demonstrated in Section 3.2, severe sparsity or extended row gaps can limit the support available for a new centerline estimate. These conditions motivate further investigation of temporal information and complementary sensing. Adapting the region of interest, classification thresholds, and RANSAC parameters to observation quality could provide additional flexibility under varying field conditions.

Alongside these methodological extensions, further validation is needed to establish closed-loop navigation performance. Evaluation on recorded field data enabled quanti-tative assessment of estimation accuracy and computational performance under repro-ducible conditions. However, no row line satisfied the acceptance criteria in 13.91% of the evaluated scans, leaving gaps in the availability of new geometric observations.

5. Conclusion

This study developed a hybrid LiDAR perception framework for agricultural crop-row navigation that integrates GroveNet-based cluster filtering with RANSAC row-line fitting and geometric reliability checks. GroveNet achieved an F1-score of 0.9768 with approximately 75K parameters. Learned filtering reduced the candidate point count by approximately 19.1% and increased the number of accepted row lines by approximately 25.9%, with lower residual dispersion. Across the evaluated straight-driving scans, the proportion containing at least one accepted row line increased from 70.0% for the geometric baseline to 86.09% for the proposed pipeline. These results support improved fitting consistency and more frequent availability of accepted row geometry.

Validation against a separately constructed reference based on georeferenced canopy geometry and RTK-derived robot positions yielded a mean absolute lateral error of 13.91 cm. The correlation of 0.90 indicated a strong positive association between the estimated and reference lateral deviations. On the NVIDIA Jetson AGX Orin, the mean processing time was 99.93 ms, below the 100 ms LiDAR scan period. These measurements support computational feasibility at the sensor rate under the tested conditions.

These findings apply to offline evaluation on straight-driving scans from a single orchard block. Further work should extend the corridor model to curved rows and assess adaptive estimation and complementary sensing under

sparse observations. Closed-loop experiments should use the estimated centerline to steer a robot and measure its actual lateral deviation from the reference centerline, particularly during interruptions in reliable row observations.

Acknowledgments: The authors gratefully acknowledge financial support from the Science Committee of the Ministry of Science and Higher Education of the Republic of Kazakhstan under research project AP23490705 (2024–2026). The authors also thank the Department of Mechanical Engineering of the Institute of Energy and Mechanical Engineering named after A. Burkitbayev, the Cybersecurity and Artificial Intelligence Research Laboratory of the Institute of Automation and Information Technologies at Satbayev University, and RnD Center LLP for their support during this research.

Conflict of Interest

The authors declare no conflicts of interest.

Supporting Information

Not applicable.

References

[1] G. Kootstra, X. Wang, P. M. Blok, J. Hemming, E. van Henten, Selective harvesting robotics: Current research, trends, and future directions, *Current Robotics Reports*, 2021, **2**, 95-104, doi: 10.1007/s43154-020-00034-1.

[2] R. Bogue, Agricultural robots: Developments and applications, *Industrial Robot*, 2023, **50**, 1-6.

[3] S. Das, E. K. Wiafe, A. Upadhyay, J. Mettler, K. Howatt, X. Sun, Design and evaluation of a vision-guided robotic platform with root-zone drilling for targeted weed removal, *Computers and Electronics in Agriculture*, 2026, **246**, 111605, doi: 10.1016/j.compag.2026.111605.

[4] A. G. C. Upadhyay, S. Y. Zhang, C. Koparan, X. Sun, Development and evaluation of a machine vision and deep learning-based smart sprayer system for site-specific weed management in row crops: An edge computing approach, *Journal of Agriculture and Food Research*, 2024, **18**, 101331, doi: 10.1016/j.jafr.2024.101331.

[5] Y. Xiong, Y. Ge, Y. Liang, S. Blackmore, Development of a prototype robot and fast path-planning algorithm for static laser weeding, *Computers and Electronics in Agriculture*, 2017, **142**, 494-503, doi: 10.1016/j.compag.2017.11.023.

[6] A. Upadhyay, Y. Zhang, C. Koparan, N. Rai, K. Howatt, S. Bajwa, X. Sun, Advances in ground robotic technologies for site-specific weed management in precision agriculture: A review, *Computers and Electronics in Agriculture*, 2024, **225**, 109363, doi: 10.1016/j.compag.2024.109363.

[7] S. Bonadies, S. A. Gadsden, An overview of autonomous crop row navigation strategies for unmanned ground vehicles, *Engineering in Agriculture, Environment and Food*, 2019, **12**, 24-31, https://doi.org/10.1016/j.eaef.2018.09.001, [8] doi: 10.1016/j.mechatronics.2004.05.005.

[8] B. Astrand, A. J. Baerveldt, A vision based row-following system for agricultural field machinery, *Mechatronics*, 2005, **15**, 251-269, doi: 10.1016/j.mechatronics.2004.05.005.

[9] K. H. Choi, S. K. Han, S. H. Han, K. H. Park, K. S. Kim, S. Kim, Morphology-based guidance line extraction for an autonomous weeding robot in paddy fields, *Computers and Electronics in Agriculture*, 2015, **113**, 266-274, doi: 10.1016/j.compag.2015.02.014.

[10] W. Winterhalter, F. V. Fleckenstein, C. Dornhege, W. Burgard, Crop row detection on tiny plants with the pattern Hough transform, *IEEE Robotics and Automation Letters*, 2018, **3**, 3394-3401, doi: 10.1109/LRA.2018.2852841.

[11] J. Xue, L. Zhang, T. E. Grift, Variable field-of-view machine vision based row guidance of an agricultural robot, *Computers and Electronics in Agriculture*, 2012, **84**, 85-91, doi: 10.1016/j.compag.2012.02.009.

[12] Q. Zhang, M. S. Chen, B. Li, A visual navigation algorithm for paddy field weeding robot based on image understanding, *Computers and Electronics in Agriculture*, 2017, **143**, 66-78, doi: 10.1016/j.compag.2017.09.008.
[13] A. Ahmadi, M. Halstead, C. McCool, Towards autonomous visual navigation in arable fields, *Proceedings of the 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, 6585-6592, doi: 10.1109/IROS47612.2022.9981299.
[14] S. Kanagasingham, M. Ekpanyapong, R. Chaihan, Integrating machine vision-based row guidance with GPS and compass-based routing to achieve autonomous navigation for a rice field weeding robot, *Precision Agriculture*, 2020, **21**, 831-855, doi: 10.1007/s11119-019-09697-z.
[15] F. B. P. Malavazi, R. Guyonneau, J. B. Fasquel, S. Lagrange, F. Mercier, LiDAR-only based navigation algorithm for an autonomous agricultural robot, *Computers and Electronics in Agriculture*, 2018, **154**, 71-79, doi: 10.1016/j.compag.2018.08.034.
[16] R. Guyonneau, F. Mercier, G. Oliveira Freitas, LiDAR-only crop navigation for symmetrical robot, *Sensors*, 2022, **22**, 8918, doi: 10.3390/s22228918.
[17] P. M. Blok, H. K. Suh, K. van Boheemen, H. J. Kim, G. Kim, Autonomous in-row navigation of an orchard robot with a 2D LiDAR scanner and particle filter with a laser-beam model, *Journal of Institute of Control, Robotics and Systems*, 2018, **24**, 726-735, doi: 10.5302/J.ICROS.2018.0078.
[18] N. T. Dang, V. H. Le, T. L. Nguyen, LiDAR-based online navigation algorithm for an autonomous agricultural robot, *Control Engineering and Applied Informatics*, 2022, **24**, 90-100, doi: 10.61416/ceai.v24i2.7773.
[19] R. Liu, F. Yandun, G. Kantor, Towards over-canopy autonomous navigation: Crop-agnostic LiDAR-based crop-row detection in arable fields, *Proceedings of the 2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, 16788-16794, doi: 10.1109/ICRA55743.2025.11127550.
[20] M. V. Gasparino, V. A. H. Higuti, A. N. Sivakumar, A. E. Velasquez, M. Becker, G. Chowdhary, CropNav: A framework for autonomous navigation in real farms, *Proceedings of the 2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, 11824-11830, doi: 10.1109/ICRA48891.2023.10160990
[21] A. N. Sivakumar, S. Modi, F. Ramos, G. Chowdhary, Learned visual navigation for under-canopy agricultural robots, *Proceedings of Robotics: Science and Systems XVII*, 2021, doi: 10.15607/RSS.2021.XVII.019.
[22] H. Isack, Y. Boykov, Energy-based geometric multi-model fitting, *International Journal of Computer Vision*, 2012, **97**, 123-147, doi: 10.1007/s11263-011-0474-7.
[23] L. Davis, *Handbook of Genetic Algorithms*, Thomson Publishing Group, Washington, DC, USA, 1991.
[24] J. F. Rada-Vilela, A fuzzy logic control library in C++, *Proceedings of the Open Source Developers Conference*, Auckland, New Zealand, 2013, 21-23.
[25] K. Chen, B. T. Lopez, A.-a. Agha-mohammadi, A. Mehta, Direct LiDAR odometry: Fast localization with dense point clouds, *IEEE Robotics and Automation Letters*, 2022, **7**, 2000-2007, doi: 10.48550/arXiv.2110.00605.
[26] G. Kim, A. Kim, Scan Context: Egocentric spatial descriptor for place recognition within 3D point cloud map, *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, Madrid, Spain, 1-5 October 2018, 4802-4809, doi: 10.1109/IROS.2018.8593953.
[27] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, J. D. Tardos, ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM, *IEEE Transactions on Robotics*, 2021, **37**, 1874-1890, doi: 10.1109/TRO.2021.3075644.
[28] Z. Teed, J. Deng, DROID-SLAM: Deep visual SLAM for monocular, stereo, and RGB-D cameras, *Advances in Neural Information Processing Systems*, 2021, **34**, 16558-16569, doi:10.48550/arXiv.2108.10869.

[29] B. Kerbl, G. Kopanas, T. Leimkuehler, G. Drettakis, 3D Gaussian Splatting for real-time radiance field rendering, *ACM Transactions on Graphics*, 2023, **42**, 139, doi: 10.1145/3592433.

[30] M. Usuelli, D. Rapado-Rincon, G. Kootstra, M. Matteucci, AgriGS-SLAM: Orchard mapping across seasons via multi-view Gaussian Splatting SLAM, *IEEE Robotics and Automation Letters*, 2026, doi: 10.1109/LRA.2026.3685453.

[31] H. Huang, L. Li, H. Cheng, S.-K. Yeung, Photo-SLAM: Real-time simultaneous localization and photorealistic mapping for monocular, stereo and RGB-D cameras, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Seattle, WA, USA, 17-21 June 2024, 21584-21593, doi: 10.48550/arXiv.2311.16728.

[32] E. Sandstrom, G. Zhang, K. Tateno, M. Oechsle, M. Niemeyer, Y. Zhang, M. Patel, L. Van Gool, M. R. Oswald, F. Tombari, Splat-SLAM: Globally optimized RGB-only SLAM with 3D Gaussians, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Nashville, TN, USA, 11-15 June 2025, 1680-1691, doi:10.48550/arXiv.2403.19549

[33] Yu, S., Cheng, C., Zhou, Y., Yang, X., & Wang, H. (2025, May). Rgb-only gaussian splatting slam for unbounded outdoor scenes. In *2025 IEEE International Conference on Robotics and Automation (ICRA)* (pp. 11068-11074). IEEE, doi: 10.1109/ICRA55743.2025.11128286.

[34] Y. Pan, X. Zhong, L. Jin, L. Wiesmann, M. Popovic, J. Behley, C. Stachniss, PINGS: Gaussian Splatting meets distance fields within a point-based implicit neural map, *arXiv*, 2025, arXiv:2502.05752, doi: 10.48550/arXiv.2502.05752.

[35] N. Habibie, A. M. Nugraha, A. Z. Anshori, M. A. Masum, W. Jatmiko, Fruit mapping mobile robot on simulated agricultural area in Gazebo simulator using simultaneous localization and mapping (SLAM), *Proceedings of the International Symposium on Micro-NanoMechatronics and Human Science*, Nagoya, Japan, 3-6 December 2017, 1-7, doi: 10.1109/MHS.2017.8305235.

[36] H. Teng, Y. Wang, X. Song, K. Karydis, Multimodal dataset for localization, mapping and crop monitoring in citrus tree farms, *Proceedings of the International Symposium on Visual Computing*, Lake Tahoe, NV, USA, 16-18 October 2023, 571-582, doi: 10.1007/978-3-031-47969-4_44.